INCUBRIX ENGINEERING ASSESSMENT 2

Prompt-Based Video Editing Engine

Edit a video by typing what you want - and keep every edit correct as the conversation goes on

Thank you for applying to IncuBrix

IncuBrix helps creators make videos with AI. In this assessment you will build one real piece of our product: the editing engine behind the IncuBrix Editor. Please read the whole brief once before you start.

At a glance	
What you build	An editing engine that turns plain-language instructions ("cut out the ums, remove the part about pricing, add a caption on the intro") into exact, reversible edits on a video, and renders the result.
Why it matters	Creators lose hours dragging clips on a timeline. Typing what they want is faster, but only if every edit stays correct when edits pile up.
What you hand over	A containerised API service, a session-runner tool, a live demo URL, your code, a short report and a test pack.
Time allowed	10 calendar days from the day you receive this brief. The exact deadline is in the email.
Tools allowed	Anything: any AI coding assistant, model, API, cloud or library. Just disclose it and keep it commercially usable.
How you are selected	You fill in a Submission Form showing what works, with proof. We verify it and run 5 hidden tests (screening, 80 marks), then hold a 60-90 minute live session (20 marks) with the strongest candidates. Shortlist: 75+. Top pick: 85+.

How to use this brief

1. Read **The problem** and **What you build** to understand the job.
2. Build the **Must-haves** first. Each one is pass or fail, and every one must pass.
3. Then climb the **Levels**: Core is required; Advanced and Outstanding earn more marks.
4. Follow the **Integration Standard** so your work can plug straight into our product.
5. Fill in the **Submission Form** (Excel file sent with this brief) as you go. It is how you show us your work, and it is the main thing we review.

The problem

A creator uploads a talking-head video (a podcast, tutorial or interview) and edits it by typing. Applying one instruction once is easy. The real problem is a **conversation of edits**: the creator keeps adding instructions, refers back to earlier ones ("make that caption bigger"), changes their mind ("actually, put the pricing part back"), is vague ("remove the boring bit"), and every edit shifts the timing that later edits depend on.

The single most important rule

Edits belong to **content**, not to a **clock position**. A caption added on the words "welcome to the channel" must stay on exactly those words after anything before it is cut, sped up or moved. If you store it as "1:40", it will silently land on the wrong words after the first cut.

What you build

- **Video intake:** accept MP4 or MOV at any common resolution and frame rate; read duration, frame rate and audio.
- **Transcript:** word-level timing for every word, with start, end, speaker and confidence. Use any speech-to-text tool, but handle its mistakes.
- **Your own timeline format:** the edited video described as data, with a JSON Schema. Designing this well is a large part of the problem.
- **Editing engine:** turns instructions into structured operations on the timeline.
- **Rendering:** produces the final video from the timeline, as a separate step.
- **Session runner:** a command-line tool that plays a scripted editing session (format below) and writes every response. We test every submission with it.

Operations to support

Cut a range; keep only a range; reorder sections; speed up or slow down a range; mute or bleep words; add a caption or text overlay; auto-caption the whole video; remove filler words (um, uh, and "like" only when used as a filler); remove silences longer than N seconds; add background music that automatically gets quieter under speech; undo and redo; restore one specific earlier edit.

Every instruction gets exactly one of three answers

Answer	What it must contain
Applied	The list of operations performed and a before/after summary (for example new length, what moved).
Needs clarification	A specific question with options, for example "Pricing is mentioned at 1:05, 2:40 and 3:15 - which one, or all?". Never guess on anything destructive.
Rejected	The reason: impossible, out of range, or contradicts an earlier edit (and which one).

Session file the runner must accept

```
video: https://.../interview.mp4
steps:
  - say: "Add a caption 'Welcome!' when I say welcome to the channel"
  - say: "Cut the first 30 seconds"
  - say: "Remove the part about pricing"
    answer: "the second one"          # reply used if the system asks a question
  - say: "Undo"
  - say: "render"
# Output: results.json with, for each step: status (applied | needs_clarification | rejected),
# operations, summary, question, options, timeline_hash, duration_s; plus the final timeline and video.
```

Must-haves (all must pass)

#	Must-have	How we verify it (you give the evidence in your form)
1	**Never damage the original.** Every edit is a step in a history applied on top of the untouched original; rendering is separate.	We check the source file hash and the history after a session.
2	**Edits stick to content.** Captions, mutes, bleeps and effects stay on their words after cuts, speed changes, reorders and restores.	Scripted sessions; we inspect timelines and rendered frames.
3	**Three-way answers.** Every instruction returns Applied, Needs clarification or Rejected, in the formats above.	Every step of every session.
4	**Predictable.** The same session always produces the same timeline. An AI model may only translate words into structured operations; all timeline logic is ordinary code you wrote.	We run each session 3 times and compare timeline_hash; we read the code.

#	Must-have	How we verify it (you give the evidence in your form)
5	**Render matches the timeline.** Audio and video in sync within 1 frame, and rendered length equals timeline length within 1 frame, after any mix of edits.	Frame-accurate measurement on rendered outputs.
6	**All operations above work**, including undo/redo (one instruction = one undo; a new edit clears redo) and restoring a specific earlier edit.	Scripted sessions.
7	**A clear time policy.** When a user says "at 1:20" after earlier cuts, your system uses one stated policy (original time or current time), everywhere, and says which one in its answer.	Scripted sessions.
8	**Integration Standard** followed, the **session runner** works, and the **live demo** works for the whole review period.	See the Integration Standard section.

Levels

Level	What you need to show
Core (required)	• All must-haves pass. • The 8-step worked example below runs exactly as described, on your own test video. • Works on one single-speaker video and one video with two or more speakers.
Advanced	• **Edits that shift each other:** captions follow their words; a caption whose words are cut is removed and the user is told; partial cuts of a captioned phrase follow a defined rule; sped-up captions keep a readable minimum (about 0.7 s). • **Ambiguity and references:** "that caption", "the boring part", "make it shorter", "put back what I removed" all ask or state an assumption with options; meaning-based search ("pricing" finds "cost") shows matches before cutting. • **Conflicts:** repeated instructions are safe; deliberate pauses survive later silence removal; impossible or whole-video cuts are rejected; overlapping speed changes follow a documented rule; multi-part instructions apply all or report exactly which parts applied. • **Two speakers:** "remove everything the guest says about their old job" removes only the guest's lines and asks about the host's lead-in question.
Outstanding	• **Imperfect data:** cuts snap to nearby silence so no word is clipped; low-confidence words are flagged, not bleeped blindly; variable frame rate and no-audio videos work or fail clearly; overlapping speech and possible speaker-label swaps produce warnings. • **Scale:** on a 60-minute video, each instruction responds in under 3 seconds (excluding first transcription and rendering). • **Export:** the timeline exports to OpenTimelineIO so other editors can open it. • **Proof:** an automated suite of at least 40 scripted scenarios with expected results runs in CI.

Worked example (your system must reproduce this behaviour)

User types	Correct behaviour
1. Add a caption 'Welcome!' when I say welcome to the channel	Applied. Caption attached to those 4 words (original time 0:42-0:44).
2. Cut the first 30 seconds	Applied. Video is 30 s shorter. Caption now plays at 0:12-0:14, still on the same words.
3. Remove the part about pricing	Needs clarification: pricing is mentioned three times, which one or all? User: the second one. Applied.
4. Speed up from 1:00 to 1:30 by 2x	Applied, stating which time policy was used.
5. Actually, put the intro back	Applied. The first 30 s return; caption back at 0:42; the pricing cut and speed change stay.
6. Undo	Reverses step 5 only. The intro is cut again.
7. Make that caption red	Applied to the Welcome! caption, the only one that exists.

User types	Correct behaviour
8. Render	Audio and video in sync, length matches the timeline, caption on the right words.

Hard cases we will test

Build and test your system against situations like these, and show the results in the Hard cases tab of your Submission Form. During screening we run **5 hidden tests** of the same kinds on your live demo. A good system handles each one sensibly and says what it did (in warnings or error messages) instead of failing silently.

- A caption added, then the exact sentence under it cut; then that cut restored (the caption should come back).
- A section reordered, then a word inside it muted; filler removal cutting into a bleeped word.
- "Remove 0:30-0:40" sent twice; "add a caption at 5:00" on a shorter video; times as "80 seconds", "a minute twenty", "the last 10 seconds".
- "Undo the second-to-last cut but keep the music"; "make it bigger" when no caption exists.
- "Remove everything after I say thanks" where thanks is said four times.
- "Speed up 10x", "cut everything", and a three-part instruction where one part is impossible.
- A two-speaker video with a speaker-label swap, overlapping speech, background music, and a phone video with variable frame rate.

How it must plug into IncuBrix (Integration Standard)

IncuBrix runs its services as containers that talk to each other over HTTP APIs. If your submission follows the ten points below, we can connect it to the product without rewriting it. Each point is worth 2 marks (20 marks in total). Tick each one in your Submission Form with a link to the proof.

Requirement	What exactly we check
1. One-command start and test	`docker compose up` starts the service and everything it needs (database, storage, local test servers). One command runs all tests (unit, integration and at least one end-to-end) with no API keys. A `.env.example` lists every setting.
2. Documented HTTP API	An `openapi.yaml` (OpenAPI 3.1) that matches the running service exactly. All routes under `/api/v1/`.
3. Background jobs	Anything slower than 2 seconds runs as a job: `POST` returns `202` with a `job_id`; `GET /api/v1/jobs/{job_id}` returns status (queued, running, succeeded, failed), progress 0-100 and result links. Optional callback URL, signed with HMAC-SHA256 in an `X-IncuBrix-Signature` header.
4. Safe retries	An `Idempotency-Key` header on every POST: sending the same request twice with the same key returns the same job and never does the work twice.
5. Clear errors	Errors are JSON: `{"error": {"code", "message", "retryable", "details"}}`. Codes are UPPER_SNAKE_CASE and listed in the README.
6. Configuration	All settings, paths, model names and limits come from environment variables or a config file. No hard-coded paths or keys.
7. Files in, files out	Inputs accepted as an upload or a URL. Outputs saved to S3-compatible storage (MinIO in the compose file is fine) and returned as download links.
8. Logs and health	Structured JSON logs with timestamp, level, job_id and the `X-Correlation-Id` from the request. `GET /health` and `GET /ready` endpoints.
9. Output manifest	Every result includes a JSON manifest: which models/providers and versions were used, settings, seed, timings, cost estimate, warnings, and licence/rights information.
10. Batch runner	A command-line tool that runs a folder or file of test inputs against the API and writes all results to one folder. We use this to test every submission the same way.

Tools and resources: use anything

We want to see the best system you can build, the way you would build it at work. There are no restrictions on tools.

- **AI coding assistants** (ChatGPT, Claude, Copilot, Cursor and so on): allowed, as much as you like.
- **Any AI model or service**: open-source or proprietary, self-hosted or hosted API, free or paid.
- **Any compute**: your laptop, a cloud GPU, a free notebook, a paid server.
- **Any open-source code or library**, as long as you credit it and its licence allows commercial use.

Because your solution should be able to go straight into the IncuBrix product, five rules apply:

1. **Disclose everything.** List every model, API, library, dataset and media source in SOURCES.md, and every AI coding tool you used (and for what) in AI_USE.md.
2. **Commercially usable.** Everything in your default path, and everything it outputs, must be licensed for commercial use. No research-only or non-commercial models, data or media (for example CC BY-NC) in the default path. Avoid AGPL code in the shipped service.
3. **Swappable providers.** Put every external AI service behind a small adapter selected by configuration, and include a mock adapter so tests run with no API keys.
4. **Know your cost.** Report the cost and time per unit of output (for example per minute of video). Paid services are fine, but you will not get extra marks for spending more. IncuBrix does not reimburse costs unless agreed in writing before you spend.
5. **No secrets in the code.** API keys live in environment variables or a secret store, never in the repository, logs or screenshots.

How we evaluate (simple, three steps)

You do most of the work of showing what you built: you fill in the **Submission Form** with what works and a link to the proof for each claim. We then check your claims and test your system ourselves.

Step	Who	What happens
1. Submission	You	Send the links and the completed Submission Form: must-haves met, levels reached, Integration Standard points done, numbers measured, hard cases tested. Every claim needs a link to evidence (a file, log, video timestamp, test or URL).
2. Screening	Us, about 45 minutes	We check that the must-haves really pass, verify your claims using your evidence and your live demo, and run 5 hidden tests. This gives a screening score out of 80.
3. Live session	You and us, 60-90 minutes	Only for candidates whose must-haves all pass and who score at least 60 of 80 in screening. Scored out of 20.

Marks (100)

Part	Marks	How the marks are given
Must-haves	Pass / fail	Every must-have must pass. One failure means the submission is not selected.
Levels reached	40	Core 15 + Advanced 15 + Outstanding 10. A level counts only when every item in it is verified, and only if the level before it is verified.
Integration Standard	20	2 marks for each of the 10 points we confirm.
Hidden tests	20	5 tests on your live demo, 4 marks each: 4 = handled correctly, 2 = handled with a clear flaw, 0 = wrong, crash or silent failure.
Live session	20	4 parts, 5 marks each (see the live session section).

Be precise and honest in your form

A claim without evidence scores 0. A claim that turns out to be false disqualifies the whole submission. It is far better to mark something "No" or "Partial" and explain why.

Numbers you must report (Metrics tab of the form)

Metric	How to measure it
Instruction latency	Median and slowest response time per instruction on a 5-minute and a 60-minute video (excluding first transcription and render).
Render accuracy	Largest A/V offset and largest length difference, in frames, across all renders.
Determinism	Share of sessions with identical timeline_hash across 3 runs (must be 100%).
Clarification quality	How many ambiguous instructions asked or stated an assumption, versus guessed.
Word clipping	Number of cuts that clip part of a word, from a listening check of 20 cuts.
Cost	Speech-to-text and language-model cost per minute of video edited.

Who we select

We are hiring only the strongest engineers, so the bar is high and the same for everyone.

Result	What it takes
Top pick - hire	All must-haves pass, total of 85 or more, Outstanding level verified, and at least 14 of 20 in the live session.
Shortlist - hire	All must-haves pass, total of 75 or more, and at least 14 of 20 in the live session.
Not selected	Any must-have fails, screening below 60 of 80, total below 75, or live session below 14.
Disqualified	Any false claim in your form, fake or edited evidence, hidden manual work presented as automated, using someone else's work without credit, using people's voice, face or content without rights, or not being able to explain your own code.

If two candidates tie, the higher live-session score wins, then hidden tests, then Integration Standard.

The live session (60-90 minutes, online)

If all your must-haves pass and you score at least 60 of 80 in screening, we invite you to a live session. You share your screen and work in your own repository and environment. You may use documentation and your usual AI coding tools. Each part is scored 5 (excellent), 3 (good), 1 (weak) or 0 (not shown).

Part	What happens	What a 5 looks like
1. Walkthrough	You explain the architecture and trace one output field from the request through to the final file.	Clear, accurate, you know why each piece exists and what would break if it changed.
2. Predict and run	We give you an input you have not seen. You predict what your system will do, then run it.	Prediction matches; when it does not, you explain why within minutes.
3. Debugging	We break something (a setting, a dependency, a provider response) and you find and fix it.	You find the cause from logs and code quickly and fix it properly, not with a workaround.
4. Live change	For example: add a new operation such as "zoom in on the speaker", or a new rule for overlapping speed changes, then run it through the session runner.	You write or update a test first or alongside, make the change, run the tests and the end-to-end path, and it works.

What to send us

Reply to the invitation email before the deadline, attaching your completed Submission Form. Put all links in the form. Keep every link working until we announce results.

Item	What it must contain
Submission Form	The Excel file sent with this brief, fully completed: every must-have, level item and Integration Standard point marked Yes / Partial / No with a link to the evidence, plus your measured numbers and hard-case results. This is the main thing we review.
Live demo URL	A working URL plus an access token if you protect it. Tell us the cold-start time if there is one. Put a rate limit on it so it cannot be abused.
Repository	Private GitHub repository with reviewer access (the username is in the email). Real commit history, not one final commit.
README.md	What it does, architecture diagram, setup in under 10 commands, how to run tests and the batch runner, all settings.
openapi.yaml	The API contract (Integration Standard point 2).
REPORT.md	Two to four pages: the approaches you considered and why you chose yours, what works, what does not and why, measured numbers, cost per unit, and what you would do next.
SOURCES.md and AI_USE.md	As described in the tools section.
Test pack	All inputs you tested with, all outputs (including failures), logs, and an index linking each claim in your report to a file.
Demo video	Unedited screen recording, at most 8 minutes: one clean run, one failure and how the system handles it, the tests running, and one design decision explained in code.
Session pack	Your scripted sessions (at least 15), their results.json files, the final timelines and rendered videos, including failures.
Timeline schema	The JSON Schema of your timeline format, with a one-page explanation of how edits stay attached to content.

Rules, rights and using your work

- **Individual work.** One candidate, one submission. You may use any tools and open-source code, but the design and the core logic must be yours and you must be able to explain all of it.
- **Rights.** Use only media, audio, data and content that you own, that is public domain, or that is licensed for commercial use. Never use a real person's voice, face or content without their consent.
- **Label AI output.** Mark generated media as AI-generated in the manifest.
- **Honesty.** Keep failed attempts in your test pack. An honest report of what does not work scores better than a polished demo that hides problems.
- **Using your work in our product.** This assessment is designed so that a strong submission can go straight into IncuBrix. Your code remains yours. If we want to use it, we will agree the terms with you in writing (licence or ownership, and compensation) before any use. By submitting, you confirm that the work is yours and that it and everything it depends on can legally be used commercially.
- **Questions.** Reply to the invitation email. We answer within one working day and share useful answers with all candidates on the same assessment, so nobody gets an unfair advantage.

Quick answers

Question	Answer
Can I use ChatGPT, Claude, Copilot or similar?	Yes. Tell us how in AI_USE.md. In the live session you must be able to explain and change the code yourself.
Can I use paid APIs or a paid GPU?	Yes. Report the cost per unit. We do not reimburse costs unless agreed in writing first.
Can I start from an open-source project?	Yes, with credit and a licence that allows commercial use. We score what you added and how well it works.
Do I need a nice user interface?	No. A simple web page or the API docs page is enough. We score the engine, the integration and the results.
What if I cannot finish everything?	Make the must-haves and the Core level solid first. Then go as far up the levels as you can. Report honestly what is missing.
Can a language model build the timeline directly?	No. It may only turn the user's words into structured operations. Applying them to the timeline must be your own deterministic code.

We look forward to seeing what you build. Good luck.

IncuBrix Engineering Team